Jogo 3D em React — modelo animado com drei

Oct 7, 2026 · @Kelwin

Visão geral

O jogo roda no navegador, feito em React com React Three Fiber, usando o modelo gerado no Tripo exportado em .glb com rig e animações. O código é todo escrito por mim; o seu trabalho é gerar o modelo e testar.

Dois modos possíveis:

- **Perseguição** — primeira pessoa dentro da casa mal-assombrada, fugindo do monstro. O seu modelo é o monstro.
- **Luta** — combate corpo a corpo, normalmente em terceira pessoa, com o modelo como lutador ou oponente.

Recomendação: começar pela perseguição. Ela exige menos animações e tolera imprecisão; a luta depende de hitboxes e timing de golpe, que é onde modelos gerados por IA mais dão trabalho.

Stack

O núcleo são três pacotes (three, fiber, drei); o resto entra conforme o modo.

Pacote	Para que serve	Modo
`vite` + `react`	Base do projeto	Ambos
`three`	Motor 3D por baixo de tudo	Ambos
`@react-three/fiber`	Three.js escrito como componentes React	Ambos
`@react-three/drei`	`useGLTF`, `useAnimations`, `PointerLockControls`, `KeyboardControls`, `PositionalAudio`, `Environment`	Ambos
`@react-three/rapier`	Física e colisão: paredes, chão, cápsula do jogador, sensores de gatilho e hitboxes	Ambos

Pacote	Para que serve	Modo
`zustand`	Estado do jogo (vida, fase, game over) fora do loop de render	Ambos
`@react-three/postprocessing`	Atmosfera: vinheta, ruído de filme, bloom	Perseguição
`three-pathfinding` ou `yuka`	Navegação do monstro pela casa (navmesh) sem atravessar paredes	Perseguição
`ecctrl` (opcional)	Controle de personagem pronto sobre o rapier	Luta
`leva`	Painel de debug: trocar animação, ajustar velocidade, luz	Desenvolvimento
`gltfjsx` (CLI, via npx)	Lê o .glb, lista as animações e gera o componente React	Desenvolvimento
`@gltf-transform/cli`	Comprime o .glb (texturas e malha)	Desenvolvimento

```
npm create vite@latest jogo -- --template react
npm i three @react-three/fiber @react-three/drei @react-three/rapier zustand
npm i @react-three/postprocessing three-pathfinding leva
```

Preparando o modelo do Tripo

Exporte em **.glb** (nunca .obj, que perde rig e animações). Antes de qualquer código, eu confiro o arquivo:

- **Quais animações existem e como se chamam** — `npx gltfjsx monstro.glb` lista tudo. Nomes genéricos ("Animation", "Take 001") são renomeados num mapa no código.
- **Uma animação por arquivo?** Funciona: carrego cada .glb e junto os clipes, desde que todos usem o mesmo esqueleto (mesmos nomes de ossos).
- **Escala e orientação** — modelo gigante ou deitado se corrige no código, sem Blender.
- **Root motion** — se a animação de andar desloca o modelo, ele "volta" no fim do ciclo. Prefira animações *in place*; senão eu travo o osso raiz.
- **Peso** — modelos de IA vêm pesados. Comprimir texturas para 1024–2048 px e a malha com meshopt/draco.

Animações mínimas por modo:

Modo	Essenciais	Desejáveis
Perseguição	idle, walk, run, attack (ou grab)	scream, look around, sair de trás da porta
Luta	idle/guarda, walk, 2–3 golpes, defesa, reação a golpe, queda/morte	esquiva, vitória, golpe especial

Animações com drei

Dois hooks resolvem o carregamento: `useGLTF` lê o .glb e `useAnimations` devolve cada animação pelo nome, pronta para tocar.

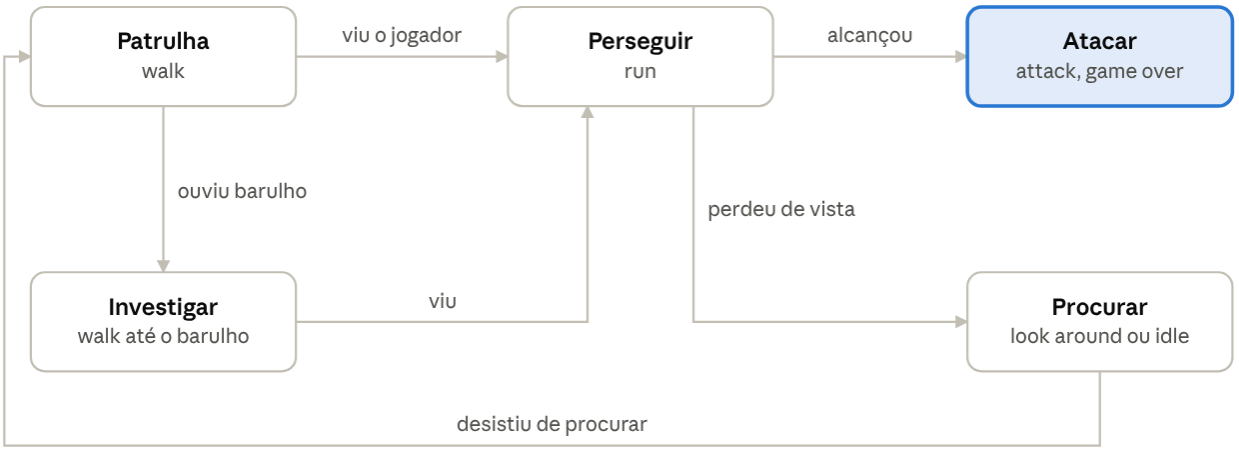
```
function Monstro({ estado }) {
  const grupo = useRef()
  const { scene, animations } = useGLTF('/monstro.glb')
  const { actions } = useAnimations(animations, grupo)

  useEffect(() => {
    const clipe = { patrulha: 'walk', perseguir: 'run', atacar: 'attack' }
[estado]
    const acao = actions[clipe]
    acao?.reset().fadeIn(0.3).play()
    return () => acao?.fadeOut(0.3)
  }, [estado])

  return <primitive ref={grupo} object={scene} />
}
```

O jogo nunca manda "tocar animação"; ele muda o **estado** do personagem, e cada estado tem a sua animação. O `fadeIn` / `fadeOut` de 0,3 s faz a transição suave em vez de "pular" de pose. Ajustes úteis: `timeScale` para acelerar a corrida, e o evento `finished` do mixer para voltar ao idle depois de um golpe.

O monstro troca de animação a cada mudança de estado



estados do monstro · 5 estados, 6 transições

Na perseguição, o monstro só ataca vindo da perseguição; perder o jogador de vista sempre passa por uma busca antes de voltar à patrulha.

Modo perseguição

O medo vem mais de som, luz e incerteza do que de gráfico. O que precisa:

- **Jogador em primeira pessoa** — `PointerLockControls` (mouse) + `KeyboardControls` (WASD), corpo como cápsula do rapier para colidir com paredes. Correr gasta fôlego e faz barulho.
- **Lanterna** — `SpotLight` presa à câmera, com bateria e falhas. É a única luz forte da cena.
- **Casa** — primeiro em blocos cinza (cômodos, portas, corredores), depois troca por modelos. A mesma geometria gera o navmesh.
- **Visão do monstro** — cone de visão + raio (raycast) até o jogador; se uma parede bloqueia, ele não vê.
- **Audição** — correr, derrubar coisa ou bater porta cria um ponto de barulho que o monstro investiga.
- **Esconderijo** — armário ou embaixo da cama tira o jogador da visão.
- **Som posicional** — passos e respiração do monstro com `PositionalAudio`, para ouvir de que lado ele vem.
- **Sustos por gatilho** — sensores do rapier em pontos da casa disparam porta batendo, vulto, objeto caindo.
- **Objetivo** — achar chaves e sair. Game over quando o ataque acerta.

Modo luta

Luta é mais exigente: o golpe precisa acertar no momento certo da animação. O que precisa além da stack base:

- **Câmera em terceira pessoa** — atrás do personagem ou de lado (estilo luta 2.5D, mais simples).
- **Hitbox e hurtbox** — sensores do rapier presos aos ossos da mão e do pé (atacam) e ao corpo (recebe). Os nomes dos ossos do Tripo eu levanto quando abrir o arquivo.
- **Janela ativa do golpe** — o hitbox só vale num trecho do clipe (ex.: de 0,30 s a 0,45 s do soco). Fora disso, encostar não conta.
- **Dano e reação** — vida, *hitstun* (travado após apanhar), empurrão para trás, defesa reduzindo dano.
- **Sensação de impacto** — pausa de ~60 ms no acerto, tremida de câmera, som e partícula. É o que separa luta boa de luta "mole".
- **Buffer de comandos** — guardar o próximo golpe apertado durante o atual, para combos.
- **IA do oponente** — decide por distância: aproxima, ataca, defende com certa chance, recua com pouca vida.
- **HUD** — barras de vida e round em React comum por cima do canvas.

Armadilhas conhecidas

- **Mais de uma cópia do modelo** — dois monstros ou dois lutadores com o mesmo .glb exigem `SkeletonUtils.clone`; um clone comum compartilha o esqueleto e os dois se mexem juntos.
- **Colisão separada da malha** — a física usa uma cápsula simples; a malha animada só acompanha. Colidir com a malha detalhada trava o jogo.
- **Animação de uma vez só** — ataque e morte precisam `LoopOnce` + `clampWhenFinished`, senão repetem em loop.
- **Pointer lock e áudio pedem clique** — o navegador só libera mouse travado e som depois de uma interação. Tela inicial com "Clique para começar".
- **Celular** — pointer lock não existe no mobile; lá precisa de joystick virtual na tela.
- **Sombras custam caro** — uma luz com sombra (a lanterna) já basta; várias derrubam o FPS.
- **Licença do modelo** — confirmar se o plano usado no Tripo permite uso comercial antes de vender o jogo.

Ordem de construção

Cada etapa termina em algo jogável ou visível; perseguição primeiro, luta depois se fizer sentido.

1. **Modelo na tela** — projeto Vite + React Three Fiber, .glb carregado, painel `leva` trocando entre todas as animações.
2. **Andar pela casa** — cenário em blocos cinza, primeira pessoa com colisão.
3. **Monstro vivo** — máquina de estados com as animações, perseguindo em linha reta.
4. **Monstro esperto** — navmesh, visão, audição, esconderijo, game over.
5. **Atmosfera** — lanterna, neblina, sons posicionais, pós-processamento.
6. **Jogo completo** — chaves, saída, sustos roteirizados, tela inicial e final.
7. **Luta (opcional)** — arena 1 contra 1 reaproveitando o modelo, hitboxes e IA de combate.